Designing with Exceptions

Your own exception classes, chaining, and how exceptions meet inheritance

Part 1 covered the mechanics: throwing, catching, and cleaning up. This part is about design. It covers which standard exceptions to use, how to write your own exception class, how to choose between checked and unchecked, how to wrap one exception inside another, and the rules that connect exceptions to the inheritance unit. It ends with habits that separate exception handling that helps from exception handling that hides problems. For reading, see Core Java, Volume I, Chapter 7.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. Use the Standard Exceptions First

Before writing a new exception class, check whether Java already has one that says what you mean. Other programmers already know what the standard ones mean, so using them makes your code easier to read.

Table 1: Standard unchecked exceptions and when to throw each.

Exception	Throw it when
`IllegalArgumentException`	An argument is not acceptable, such as a negative salary.
`IllegalStateException`	The object is not in a state where the method makes sense, such as posting to a ledger that is already closed.
`NullPointerException`	An argument that must not be `null` is `null`. `Objects.requireNonNull(name, "name is required")` checks and throws in one line.
`IndexOutOfBoundsException`	An index is outside the valid range.
`UnsupportedOperationException`	A method exists, often because an interface requires it, but this class does not support it.

You do not usually throw `NumberFormatException` yourself. Methods such as `Double.parseDouble` throw it for you, and your code decides what to do about it.

2. Writing Your Own Exception Class

Sometimes no standard exception fits, or the caller needs extra information that a message string cannot carry well. Then you write your own exception class. It is an ordinary class that extends `Exception` (for a checked exception) or `RuntimeException` (for an unchecked one).

Here is an exception for a line in a staff file that cannot become an employee. It carries the line number as well as a message:

```
/**
 * Thrown when one line of a staff file cannot be turned into an employee.
 * It is checked, because a bad line in a file is something the caller can
 * expect and recover from, for example by skipping the line.
 */
public class BadRecordException extends Exception {
    private static final long serialVersionUID = 1L;

    private final int lineNumber;

    public BadRecordException(int lineNumber, String message) {
        super("line " + lineNumber + ": " + message);
        this.lineNumber = lineNumber;
    }

    public BadRecordException(int lineNumber, String message, Throwable cause) {
        super("line " + lineNumber + ": " + message, cause);
        this.lineNumber = lineNumber;
    }

    public int getLineNumber() {
        return lineNumber;
    }
}
```

Everything in this class comes from earlier units:

- **Constructor chaining.** Each constructor passes the message up with `super(...)`, so `getMessage()`, inherited from `Throwable`, returns it.
- **Two constructors by convention.** One takes a message. The other also takes a *cause*, which Section 4 explains.
- **A final field.** The line number never changes once the exception exists.
- **`serialVersionUID`.** `Throwable` implements `Serializable`, so every exception class is serializable, which is the serialization unit showing up again. Without this field, the compiler warns when warnings are on, and Eclipse shows the same warning:

```
warning: [serial] serializable class NoSerial has no definition of serialVersionUID
```

Name your exception class for the problem, and end the name with `Exception`.

3. Checked or Unchecked?

This is a design decision, and reasonable programmers disagree about it. These questions help.

Table 2: Questions that point toward a checked or an unchecked exception.

Question	If yes	If no
Could a correct program still run into this, because of something outside the code?	Checked	Unchecked
Can the caller reasonably do something about it, such as skip a line or ask again?	Checked	Unchecked
Would a careful programmer prevent it by checking first, as with a null argument?	Unchecked	Checked

A bad line in a data file passes the first two questions: correct code cannot prevent a bad file, and the caller can skip the line. So `BadRecordException` is checked. A negative salary passed to a constructor is a programming mistake in the calling code, so `IllegalArgumentException`, which is unchecked, fits.

Many modern libraries lean toward unchecked exceptions, because checked exceptions force every method in a chain to declare them. Java's own file classes use checked exceptions. In this course, either choice can be right. What matters is that the choice is deliberate and you can explain it.

4. Translating and Chaining Exceptions

Low-level code throws low-level exceptions. When `Double.parseDouble` fails, it throws a `NumberFormatException`, which says nothing about staff files or line numbers. A good method catches the low-level exception and throws one that makes sense at its own level, passing the original along as the *cause*:

```
record Staff(String name, double salary) { }

static Staff parse(int lineNumber, String line) throws BadRecordException {
    String[] parts = line.split(",");
    if (parts.length != 2) {
        throw new BadRecordException(lineNumber, "expected name,salary but found \"" +
    line + "\"");
    }
    try {
        double salary = Double.parseDouble(parts[1]);
        if (salary < 0) {
            throw new BadRecordException(lineNumber, "negative salary " + salary);
        }
        return new Staff(parts[0], salary);
    } catch (NumberFormatException e) {
        throw new BadRecordException(lineNumber, "salary is not a number", e);
    }
}
```

This is called *exception translation*, and keeping the original as the cause is called *chaining*. If the new exception is never caught, the stack trace shows both, with the cause under "Caused by":

```
Exception in thread "main" BadRecordException: line 2: salary is not a number
        at StaffLoader.parse(StaffLoader.java:19)
        at ChainTrace.main(ChainTrace.java:3)
Caused by: java.lang.NumberFormatException: For input string: "abc"
        at java.base/jdk.internal.math.FloatingDecimal.readJavaFormatString(...)
        at java.base/java.lang.Double.parseDouble(Double.java:792)
        at StaffLoader.parse(StaffLoader.java:13)
        ... 1 more
```

The JDK's own internal lines are shortened here. The top tells you what went wrong in terms of the program, and the "Caused by" section tells you exactly where it started. Code that catches the exception can reach the original with `e.getCause()`. Never translate an exception without passing the cause, or the second half of that trace is lost.

5. After the Catch: Continue or Stop?

Catching an exception is only half the job. Every `catch` block is a design decision about what the program does next. One of the most important purposes of exception handling is to let a program either keep going safely or stop cleanly, instead of crashing with a stack trace that means nothing to the person using it. Three questions help you decide.

- Can the program still produce a correct result without the piece that failed?
- Would continuing quietly produce output that is wrong or misleading?
- Is there a sensible recovery, such as skipping a record, asking the user again, trying again, or using a default value?

Continue gracefully

If one line of a staff file is bad, the rest of the file is still useful. The program can skip that line and keep going, as the next section shows. Continuing gracefully also means being honest about it: the program should report what it skipped, so that partial results are never mistaken for complete ones.

End gracefully

If the staff file itself is missing, there is nothing to process, and no recovery makes sense. The program should stop, but it should stop cleanly: with a message the user can act on, with the technical details recorded somewhere useful (Part 4 covers logging), and with any open files closed, which try-with-resources does for you.

```
public static void main(String[] args) {
    String fileName = args.length > 0 ? args[0] : "files/staff.csv";
    List<String> lines;
    try {
```

```
        lines = Files.readAllLines(Path.of(fileName));
    } catch (NoSuchFileException e) {
        System.out.println("Could not find " + fileName + ". Check the file name and
    try again.");
        return;
    } catch (IOException e) {
        System.out.println("Could not read " + fileName + ": " + e.getMessage());
        return;
    }
    System.out.println("Loaded " + lines.size() + " lines from " + fileName);
}
```

```
Could not find files/staf.csv. Check the file name and try again.
```

The `return` statements end `main`, so the program stops after a clear message instead of a stack trace.

The danger in between

The worst choice is to continue after a failure that left the program's data half-finished, such as a payroll where some employees were paid and others were not. That produces results that look correct and are not. When you cannot be sure that continuing is safe, stop cleanly.

6. One Bad Record Should Not Stop the Program

Where you put the `try` block decides what the program does after a problem. Here the `try` block is *inside* the loop, so a bad line is recorded and the loop moves on:

```
List<String> lines = List.of("Ada,60000", "Grace,abc", "Alan,-5", "Linus", "Barbara
    ,84000");
ArrayList<Staff> staff = new ArrayList<>();
ArrayList<String> errors = new ArrayList<>();
for (int i = 0; i < lines.size(); i++) {
    try {
        staff.add(parse(i + 1, lines.get(i)));
    } catch (BadRecordException e) {
        errors.add(e.getMessage());
    }
}
System.out.println("Loaded: " + staff);
System.out.println("Skipped:");
for (String error : errors) {
    System.out.println("  " + error);
}
```

```
Loaded: [Staff[name=Ada, salary=60000.0], Staff[name=Barbara, salary=84000.0]]
Skipped:
  line 2: salary is not a number
  line 3: negative salary -5.0
  line 4: expected name,salary but found "Linus"
```

If the `try` block surrounded the whole loop instead, the first bad line would end the loop, and Barbara would never be loaded. Neither placement is always right. The question is which code knows what to do. `parse` knows a line is bad, but not what the program wants to do about it, so it throws. The loop knows the program wants to keep going, so it catches. This is the habit called *throw early, catch late*.

7. Exceptions and Inheritance

Exceptions follow the substitution principle from the polymorphism notes. Here is an interface whose method may throw an `IOException`:

```
public interface ReportWriter {
    void write(String text) throws IOException;
}
```

A class that implements it, or a subclass that overrides a method, may throw *fewer* checked exceptions or *narrower* ones, but never broader ones:

```
public class ConsoleWriter implements ReportWriter {
    @Override
    public void write(String text) {          // throws nothing: allowed
        System.out.println(text);
    }
}

public class FileReportWriter implements ReportWriter {
    @Override
    public void write(String text) throws FileNotFoundException {   // narrower:
    allowed
        try (PrintWriter out = new PrintWriter("report.txt")) {
            out.println(text);
        }
    }
}

public class BroadWriter implements ReportWriter {
    @Override
    public void write(String text) throws Exception {   // broader: does not compile
        System.out.println(text);
    }
}
```

```
error: write(String) in BroadWriter cannot implement write(String) in ReportWriter
  overridden method does not throw Exception
```

The reason is the declared type. Code holding a `ReportWriter` variable was written to handle an `IOException`, and nothing else. If an object in that variable could throw any `Exception`, that code

would be caught unprepared. A subclass must be usable anywhere its superclass is expected, and that includes the exceptions it can throw.

The rule applies only to checked exceptions. Any method may throw an unchecked exception, such as an `IllegalArgumentException`, whether or not the method it overrides does.

Constructors

Constructors cannot be overridden, but constructor chaining creates a similar rule. If a superclass constructor declares a checked exception, every subclass constructor that calls it must declare it too:

```
public class Base {
    public Base() throws IOException { ... }
}

public class Derived extends Base {
    public Derived() {              // does not compile
        super();
    }
}
```

```
error: unreported exception IOException; must be caught or declared to be thrown
        super();
             ^
```

The subclass cannot catch it instead, because the call to `super(...)` cannot be placed inside a `try` block. The fix is `public Derived() throws IOException`.

A constructor that throws also means the object is never created. That is exactly what you want from validation: the payroll constructors that reject a negative salary guarantee that no employee with a negative salary can exist.

8. Good Habits

- **Do not swallow exceptions.** An empty `catch` block hides the problem and makes the program fail later, somewhere else, for no visible reason. If you truly mean to ignore an exception, say why in a comment.

  ```
  try {
      staff.add(parse(i + 1, lines.get(i)));
  } catch (BadRecordException e) {
      // nothing here: the bad line simply disappears, and no one ever knows
  }
  ```

- **Catch the most specific type you can handle.** Catching `Exception` also catches every bug that happens to throw a `RuntimeException`, and treats it like the problem you expected. Broad catches belong only at the very top of a program, where the goal is a clean error message instead of a crash.

- **Do not use exceptions for control flow.** As Part 1 showed, decisions belong in `if` statements and loops should end by their own conditions. Checking `scanner.hasNextInt()` is clearer and faster than calling `nextInt()` and catching the failure. The bad-record loop above is not control flow: the loop still ends because of its own condition, and the `catch` block only handles a bad line, which is a genuinely exceptional event. Exceptions are for exceptional circumstances.
- **Throw early, catch late.** Throw as soon as you detect the problem, with a message that says what was wrong. Catch where the code knows what to do about it.
- **Write useful messages.** `"negative salary: -5000.0"` helps. `"error"` does not. Include the value that caused the problem.
- **Keep the cause.** When you translate an exception, pass the original to the new one's constructor.
- **Prefer try-with-resources** to closing things yourself in `finally`.

9. Summary

Table 3: Exception design decisions in brief.

Decision	Guideline
Which exception?	A standard one if it fits. Your own class if the caller needs a distinct type or extra information.
Checked or unchecked?	Checked for problems a correct program can still meet and the caller can recover from. Unchecked for programming mistakes.
Translate?	Yes, when a low-level exception would not make sense to your caller. Always keep the cause.
Where to catch?	Where the code knows what to do next, which is often a loop or the top of the program.
Overriding	An overriding method may throw fewer or narrower checked exceptions, never broader ones.

10. Try It

1. Change `BadRecordException` so it extends `RuntimeException`. Which lines of the loader no longer need to change for the program to compile? Is the program better or worse for it, and why?
2. Remove `e` from the translation in `parse`, so the cause is not passed. Run `ChainTrace` again. What information is missing from the stack trace?
3. Move the `try` block in the loader so it surrounds the whole `for` loop. Predict the output before you run it.